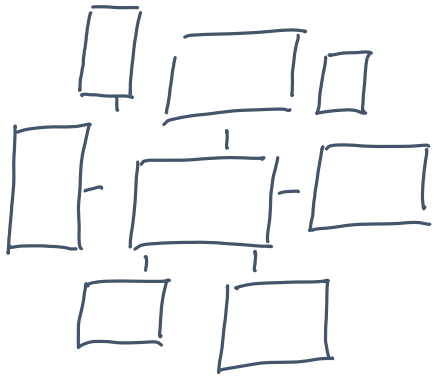


FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

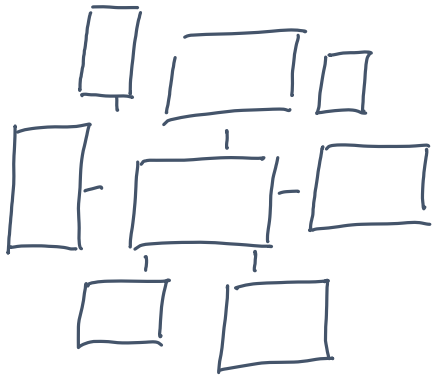
fbeit

LFSRs – Basic Concepts and Applications

Today's Agenda

Intended topics for today's session

- LFSRs – Basic Concepts and Applications
- LFSR for BIST and Test-Pattern Generation



LFSRs – Basic Concepts and Applications

International Master of Science in Electrical Engineering

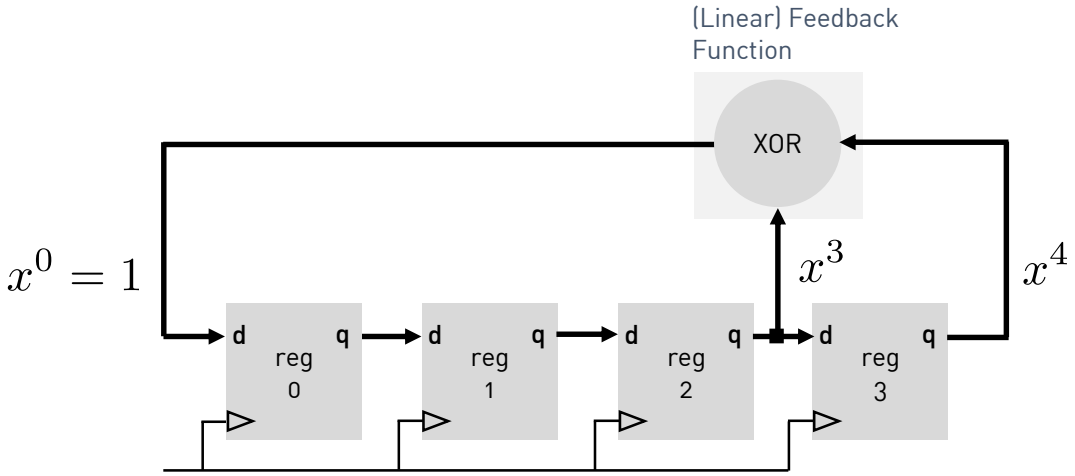
Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt
h_da
Faculty of Electrical Engineering and Information Technology
fbeit

LFSR Applications

- Test Pattern Generators and Output Response Analyzer (Built-in Self-Test (BIST))
- Counters
- Encryption
- Compression
- Checksums
- Generation of Pseudo-Random Bit Sequences (PRBS)

XOR based 4-bit Fibonacci LFSR



$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

Notes

- The all zero's state represents the lock-up state

$$P(x) = x^4 + x^3 + 1$$

Characteristic Polynomial

Leonardo Fibonacci
1170-1240
(Image Source: wikipedia.com)

Notes

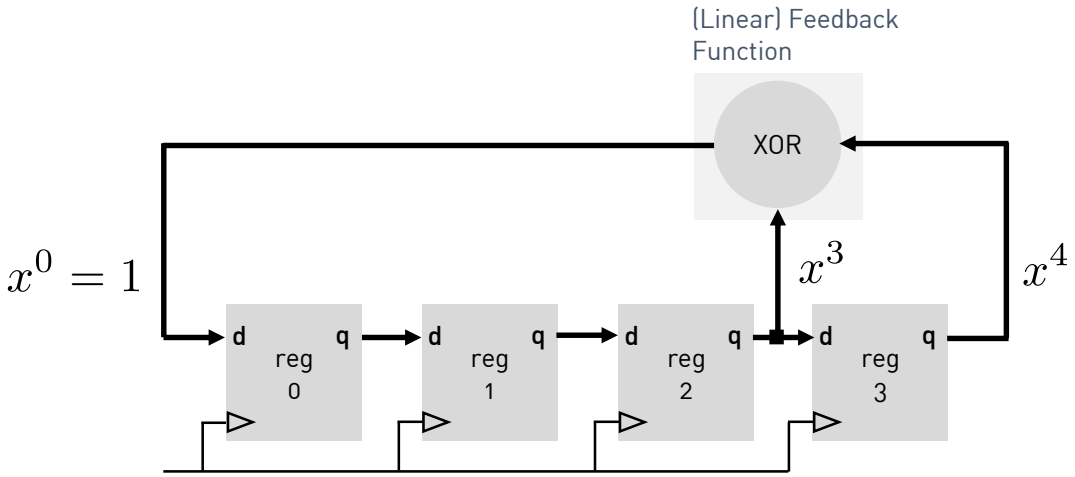
- The characteristic polynomial is one way to describe an LFSR structure
- The polynomial is defined by the position of the XOR gates
- The degree of an LFSR polynomial is defined by its highest power and is equivalent to the number of flip-flops in the circuitry
- XOR feedback connection to the i^{th} FF is equivalent to the i^{th} coefficient
  - coefficient = 0 if no connection is present
  - coefficient = 1 if a connection is present
- There are two coefficients which are always included:
  - x^n degree of the polynomial and primary feedback
  - x^0 principle input to the shift register …

XOR based 4-bit Fibonacci LFSR

Introduction to SystemVerilog

- Let's look at the generated output sequence ...

XOR based 4-bit Fibonacci LFSR



$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

Notes

- The all zero's state represents the lock-up state

$$P(x) = x^4 + x^3 + 1$$

Characteristic Polynomial

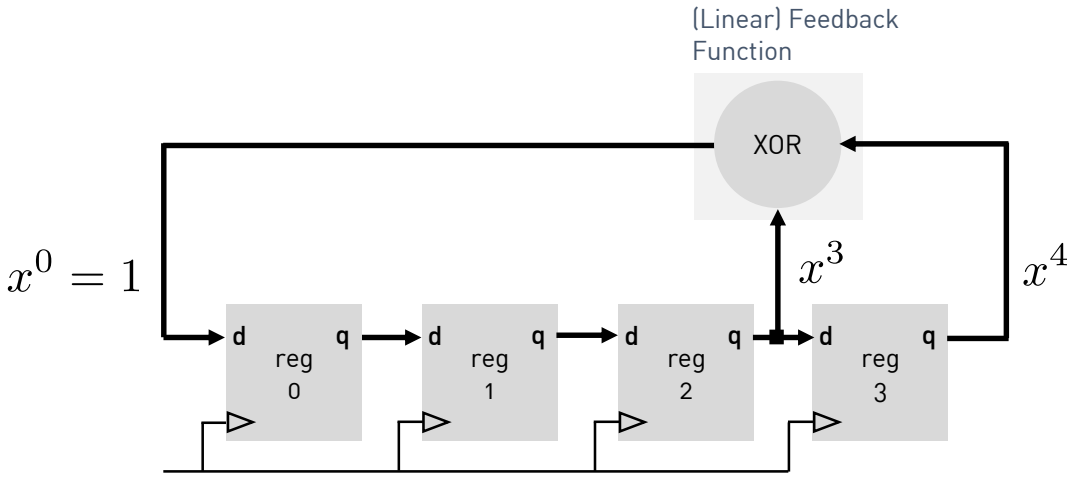
Notes

- An LFSR generates a pseudo random periodic sequence
- The maximum length of an n-bit LFSR sequence is:

$$2^n - 1$$

- In case of an XOR based LFSR, the LFSR must start in a non-zero state, therefore the state sequence contains all possible bit patterns except the all 0s pattern
- The characteristic polynomial of an LFSR generating a maximum-length sequence is known as a **primitive polynomial** (more on that later)

XOR based 4-bit Fibonacci LFSR



$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

Seed				
1	1	1	1	15
0	1	1	1	7
0	0	1	1	3
0	0	0	1	1
1	0	0	0	8
0	1	0	0	4
0	0	1	0	2
1	0	0	1	9
1	1	0	0	12
0	1	1	0	6
1	0	1	1	11
0	1	0	1	5
1	0	1	0	10
1	1	0	1	13
1	1	1	0	14
1	1	1	1	15
0	1	1	1	7

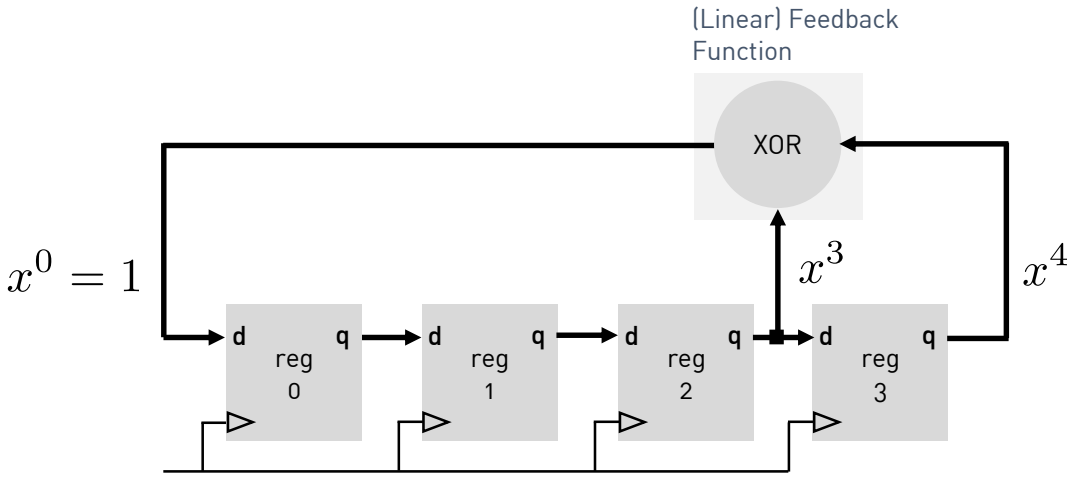
Notes

- LFSR with the characteristic polynomial:
  $$P(x) = x^4 + x^3 + 1$$
- Beginning in the all 1s state
- Maximum sequence length:
  $$2^4 - 1 = 15$$
- Polynomial is primitive

Notes

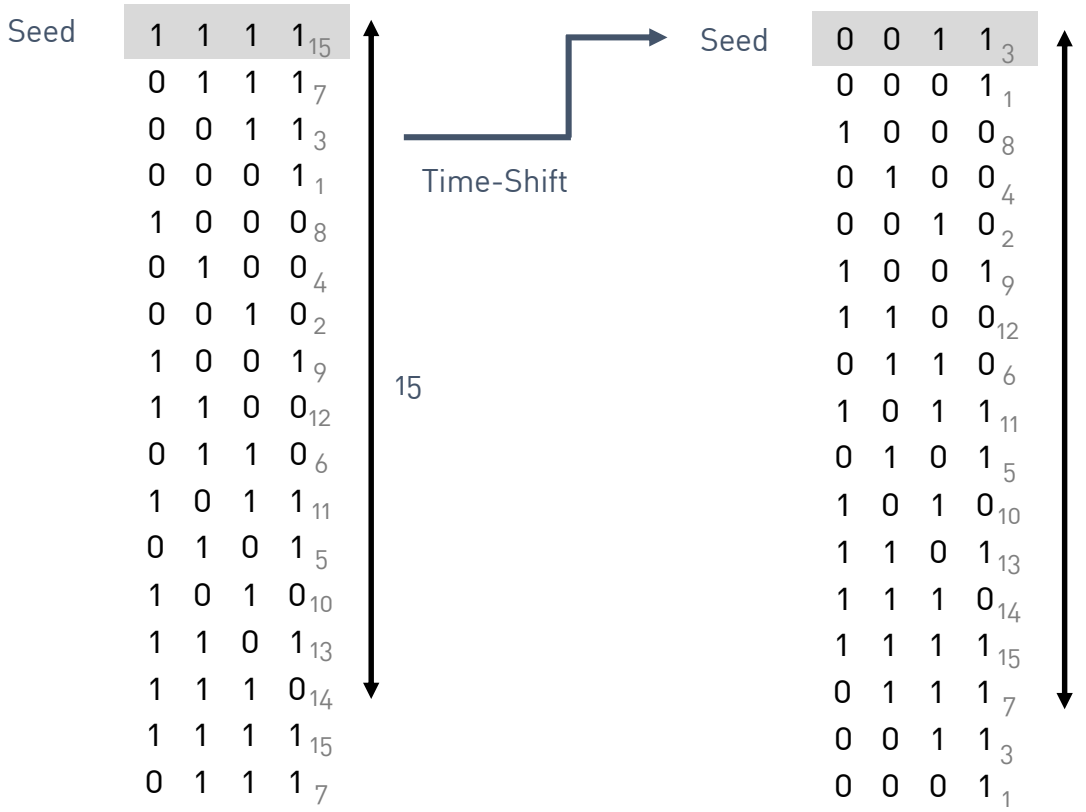
- The all zero's state represents the lock-up state

XOR based 4-bit Fibonacci LFSR



$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)



Notes

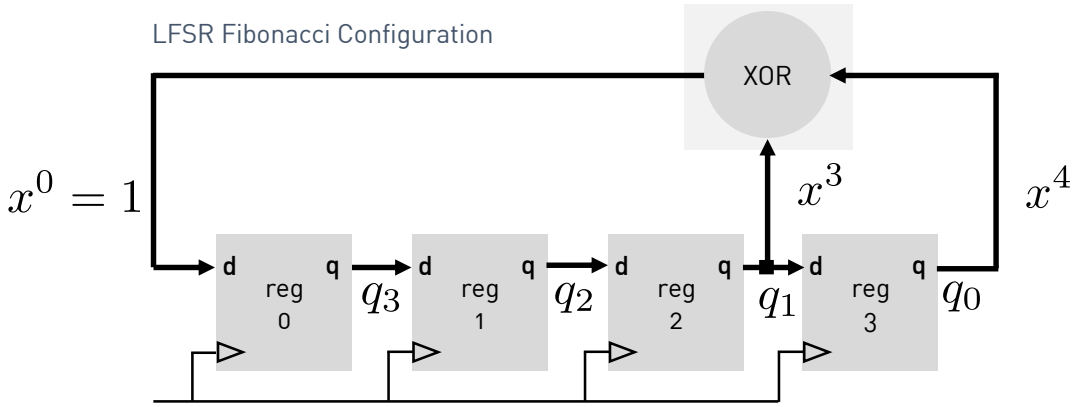
- We are just starting at another position ...

XOR based 4-bit Fibonacci LFSR

1/1

```
1.  module lfsr_fibonacci_4bit(
2.     input logic clk, input logic reset_n,
3.     output logic[3:0] q
4.  );
5.
6.  logic feedback; assign feedback = q[0]^q[1];
7.
8.  always_ff@(posedge clk)
9.     if(reset_n == 0)
10.       q <= 4'b1111;
11.    else
12.       q <= {feedback,q[3:1]};     // right-shift
13.
14. endmodule
```

[SystemVerilog] Source Code: lfsr_fibonacci_xor_4bit.sv

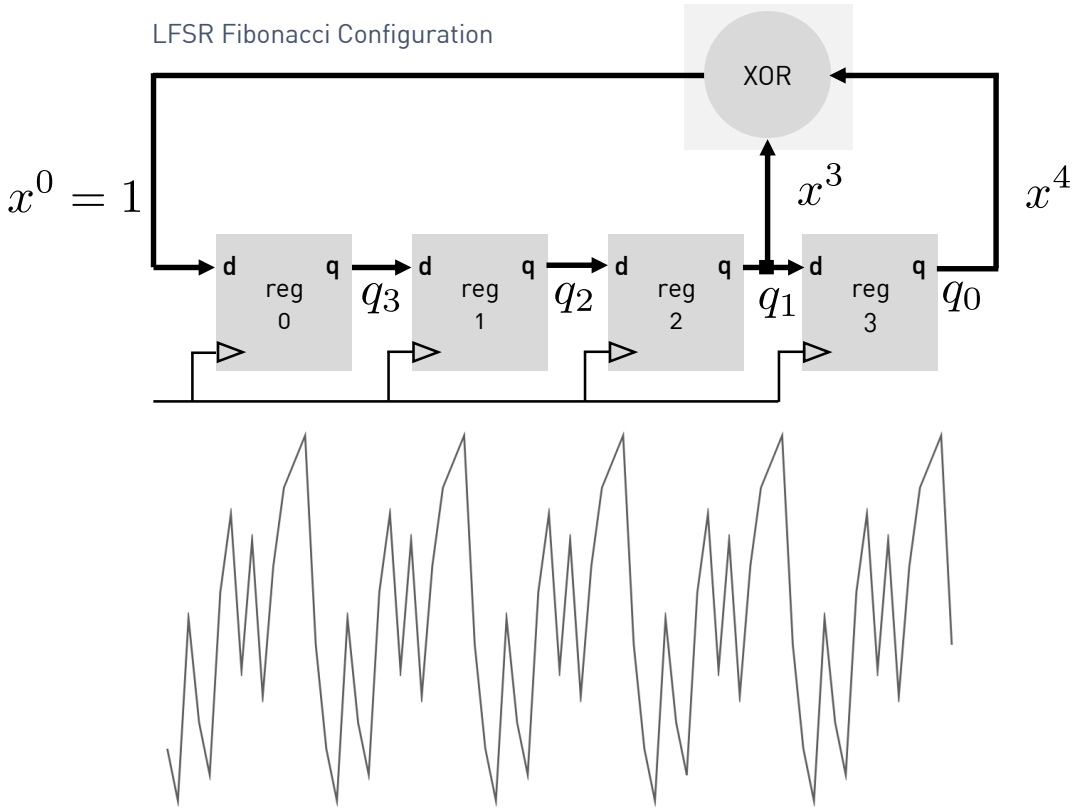


XOR based 4-bit Fibonacci LFSR

1/1

```
1.  module lfsr_fibonacci_4bit(
2.     input logic clk, input logic reset_n,
3.     output logic[3:0] q
4.  );
5.
6.  logic feedback; assign feedback = q[0]^q[1];
7.
8.  always_ff@(posedge clk)
9.     if(reset_n == 0)
10.       q <= 4'b1111;
11.    else
12.       q <= {feedback,q[3:1]};     // right-shift
13.
14. endmodule
```

[SystemVerilog] Source Code: lfsr_fibonacci_xor_4bit.sv



Pseudo 'analog' representation of the discrete LFSR output

Quiz

Notes

- By knowing the Fibonacci LFSR polynomial, we can easily sketch the corresponding circuitry ….

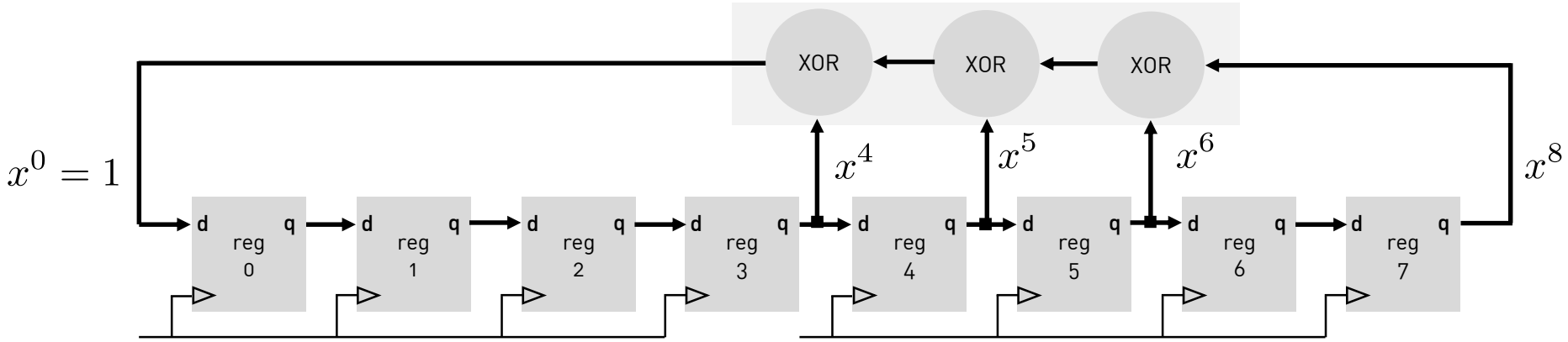
$$P(x) = x^8 + x^6 + x^5 + x^4 + 1$$

8 taps (FF)

$$P(x) = x^8 + x^6 + x^5 + x^4 + 1$$

3 XOR gates

- How many taps?
- How many XOR gates?



What about LFSR's which are not based on a primitive polynomial?

Introduction to SystemVerilog

XOR based 4-bit Fibonacci LFSR

1/1

```
1.  module lfsr_fibonacci_4bit(
2.     input logic clk, input logic reset_n,
3.     output logic[3:0] q
4.  );
5.  // non-primitive polynomial
6.  logic feedback; assign feedback = q[0]^q[2];
7.
8.  always_ff@(posedge clk)
9.     if(reset_n == 0)
10.       q <= 4'b1111;
11.    else
12.       q <= {feedback,q[3:1]};     // right-shift
13.
14. endmodule
```

[SystemVerilog] Source Code: lfsr_fibonacci_xor_4bit.sv

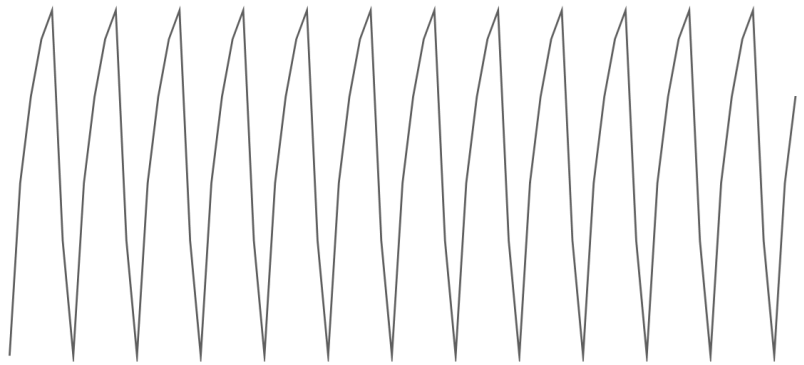
XOR based 4-bit Fibonacci LFSR

1/1

```
1.  module lfsr_fibonacci_4bit(
2.     input logic clk, input logic reset_n,
3.     output logic[3:0] q
4.  );
5.  // non-primitive polynomial
6.  logic feedback; assign feedback = q[0]^q[2];
7.
8.  always_ff@(posedge clk)
9.     if(reset_n == 0)
10.       q <= 4'b1111;
11.    else
12.       q <= {feedback,q[3:1]};     // right-shift
13.
14. endmodule
```

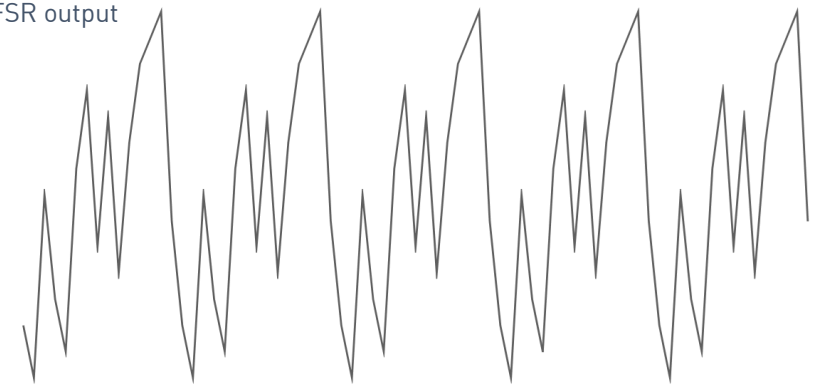
[SystemVerilog] Source Code: lfsr_fibonacci_xor_4bit.sv

$$P(x) = x^4 + x^2 + 1$$

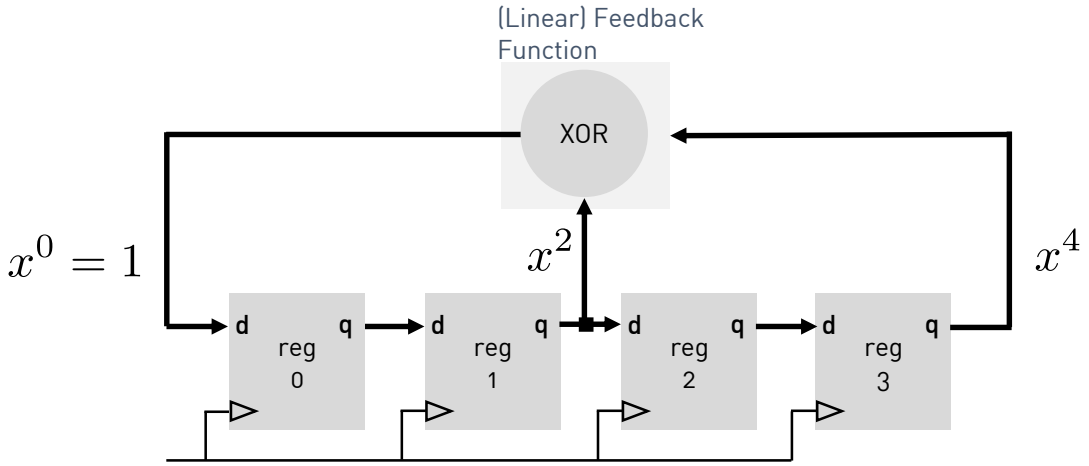


Pseudo 'analog' representation of the discrete LFSR output

$$P(x) = x^4 + x^3 + 1$$



XOR based 4-bit Fibonacci LFSR



$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

Seed				
1	1	1	1_{15}	
0	1	1	1_{7}	
0	0	1	1_{3}	5
1	0	0	1_{9}	
1	1	0	0_{12}	
1	1	1	1_{15}	
0	1	1	1_{7}	
0	0	1	1_{3}	
1	0	0	1_{9}	
1	1	0	0_{12}	
1	1	1	1_{15}	
0	1	1	1_{7}	
0	0	1	1_{3}	
1	0	0	1_{9}	
1	1	0	0_{12}	

Notes

- Example LFSR with the characteristic polynomial:

$$P(x) = x^4 + x^2 + 1$$

- Beginning in the all 1s state
- Maximum sequence length:

$$5 \neq 2^4 - 1 = 15$$

- Polynomial is not primitive
- Non-primitive polynomials produce sequences

$$< 2^n - 1$$

Notes

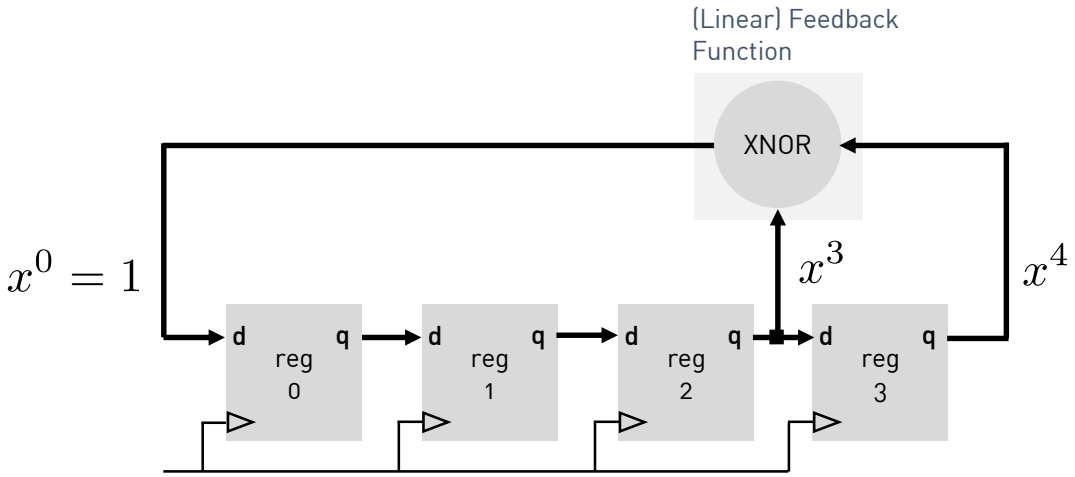
- The all zero's state represents the lock-up state

LFSR circuits can also be built equivalently with XNOR states ...

Introduction to SystemVerilog

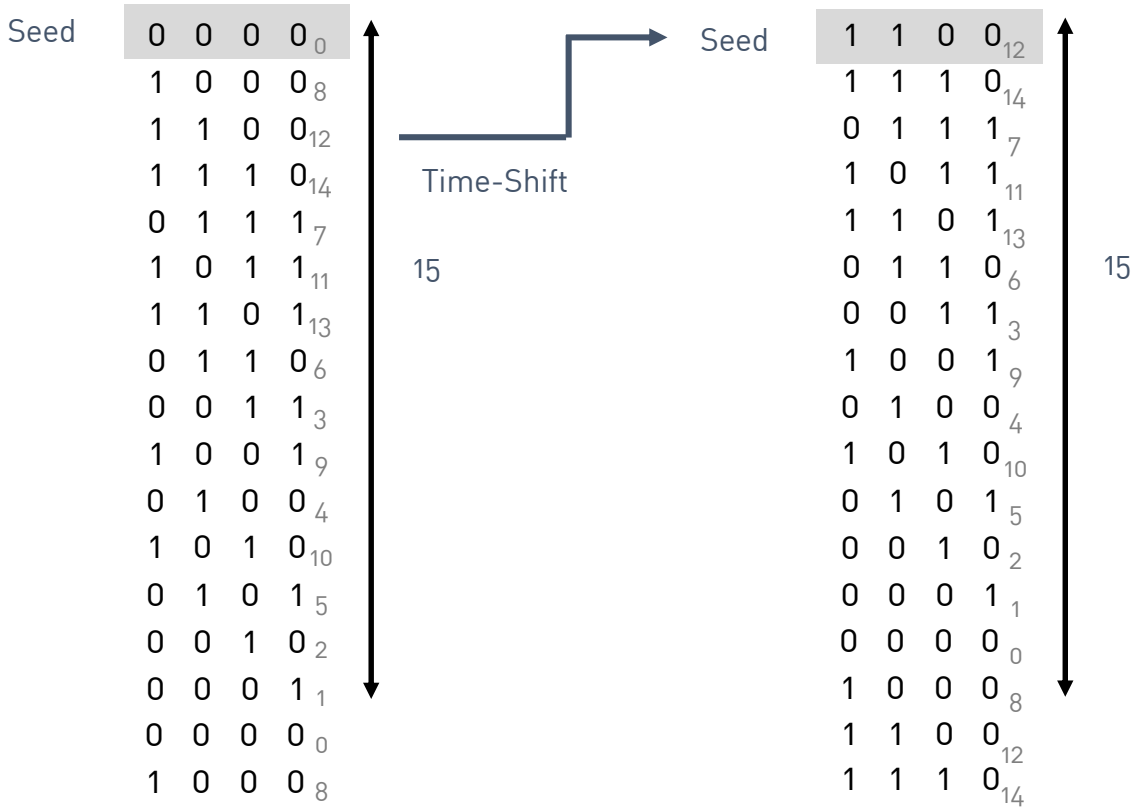
- LFSR circuits can also be built equivalently with **XNOR** gates, with the lock state being all ‘1’s instead of all ‘0’s.

XNOR based 4-bit Fibonacci LFSR



$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)



Notes

- The all **one's state** represents the lock-up state

XNOR based 4-bit Fibonacci LFSR

1/1

```
1.  module lfsr_fibonacci_4bit(
2.     input logic clk, input logic reset_n,
3.     output logic[3:0] q
4.  );
5.  // XNOR based feedback-function
6.  logic feedback; assign feedback = q[0]~^q[1];
7.
8.  always_ff@(posedge clk)
9.     if(reset_n == 0)
10.       q <= 4'b1001;
11.    else
12.       q <= {feedback,q[3:1]};     // right-shift
13.
14. endmodule
```

[SystemVerilog] Source Code: lfsr_fibonacci_xor_4bit.sv

Pseudo 'analog' representation of the discrete LFSR output

XNOR based 4-bit Fibonacci LFSR

1/1

```
1.  module lfsr_fibonacci_4bit(
2.     input logic clk, input logic reset_n,
3.     output logic[3:0] q
4.  );
5.  // XNOR based feedback-function
6.  logic feedback; assign feedback = q[0]~^q[1];
7.
8.  always_ff@(posedge clk)
9.     if(reset_n == 0)
10.       q <= 4'b1001;
11.    else
12.       q <= {feedback,q[3:1]};     // right-shift
13.
14. endmodule
```

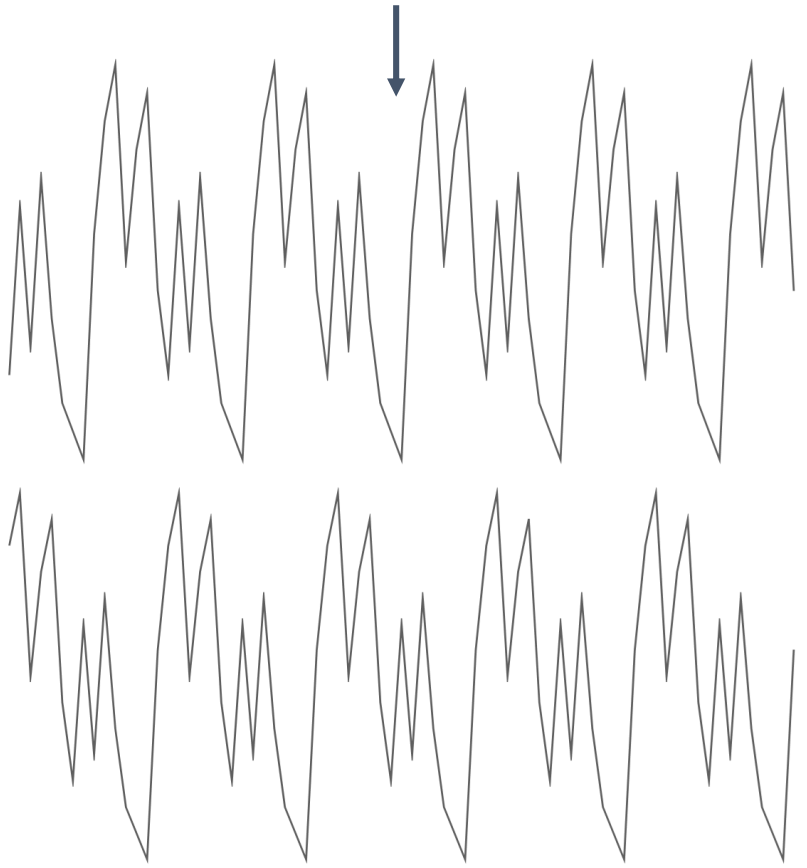
[SystemVerilog] Source Code: lfsr_fibonacci_xor_4bit.sv

$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci **XOR** Configuration

$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci **XNOR** Configuration



Pseudo 'analog' representation of the discrete LFSR output

Something in between - Why are XOR or XNOR operations linear operations?

Introduction to SystemVerilog

fbeit
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

Why are XOR or XNOR operations linear operations?

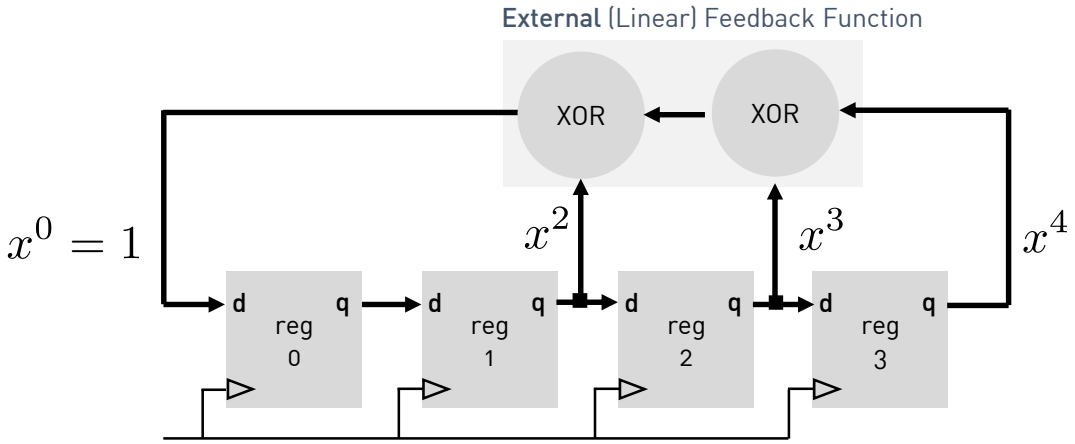
- In case of a Boolean XOR/XNOR function, the state change of one input is always causing an output change, independent of the fixed state of the other input.
- If you compare all 6 binary operations, you'll see, that this property doesn't apply for AND, OR, NAND, NOR but is valid for **XOR** - and **XNOR** operations ...

Galois LFSR Structures ...?

Introduction to SystemVerilog

fbeit
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

XOR based 4-bit Fibonacci LFSR



$$P(x) = x^4 + x^3 + x^2 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

$$P(x) = x^4 + x^3 + x^2 + 1$$

Characteristic Polynomial

Notes

- One way of implementing LFSRs; all XOR gates are fed sequentially into one another and end up as the input to the most significant bit of the LFSR. **Simply put, the XORs are external from the shift register.**

Notes

- The all zero state represents the lock-up state

Fibonacci / Galois conversion

- For a given Fibonacci LFSR, described by the polynomial P(x), the corresponding Galois LFSR representation

Notes

- Assume the following Fibonacci LFSR polynomial

$$P(x) = x^4 + x^3 + x^2 + 1$$

- In order to convert this polynomial to an equivalent Galois representation, adapt the respective exponents as follows

$$P(x) = x^{4-4} + x^{4-3} + x^{4-2} + x^{4-0}$$

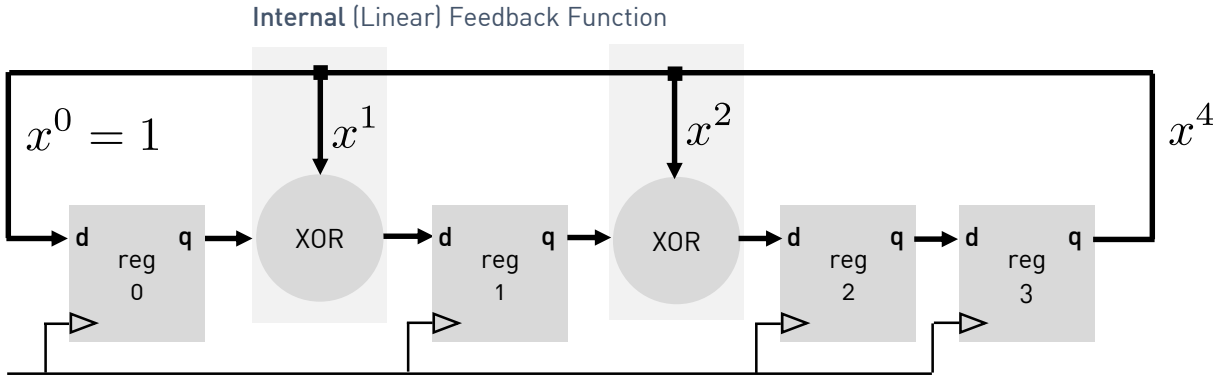
$$P(x) = x^0 + x^1 + x^2 + x^4$$

- Rearranging the previous expression leads to

$$P(x) = x^4 + x^2 + x^1 + x^0 = x^4 + x^2 + x^1 + 1$$

XOR based 4-bit Galois LFSR

Évariste Galois , 1811-1832
(Image Source: wikipedia.com)



$$P(x) = x^4 + x^2 + x^1 + 1$$

Characteristic Polynomial

Notes

- Another way to implement an LFSR is to place the XOR gates inside the shift register. **This alternate structure generates an output stream with identical sequence length**

What's the advantage of a Galois LFSR structure?

XOR based 4-bit Galois LFSR

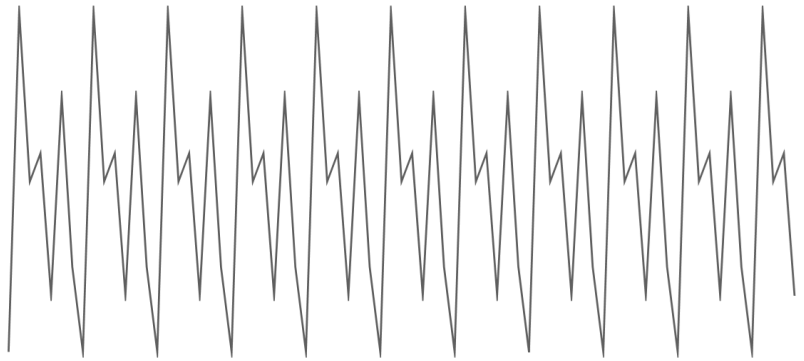
1/1

```
1.  module lfsr_galois_4bit(
2.     input logic clk, input logic reset_n,
3.     output logic[3:0] q
4.  );
5.
6.  always_ff@(posedge clk)
7.     if(reset_n == 0)
8.        q <= 4'b1001;
11.    else
12.       q <= {q[0],q[3]^q[0],q[2]^q[0],q[1]};
13.
14. endmodule
```

[SystemVerilog] Source Code: lfsr_galois_4bit.sv

$$P(x) = x^4 + x^2 + x^1 + 1$$

Galois LFSR Configuration



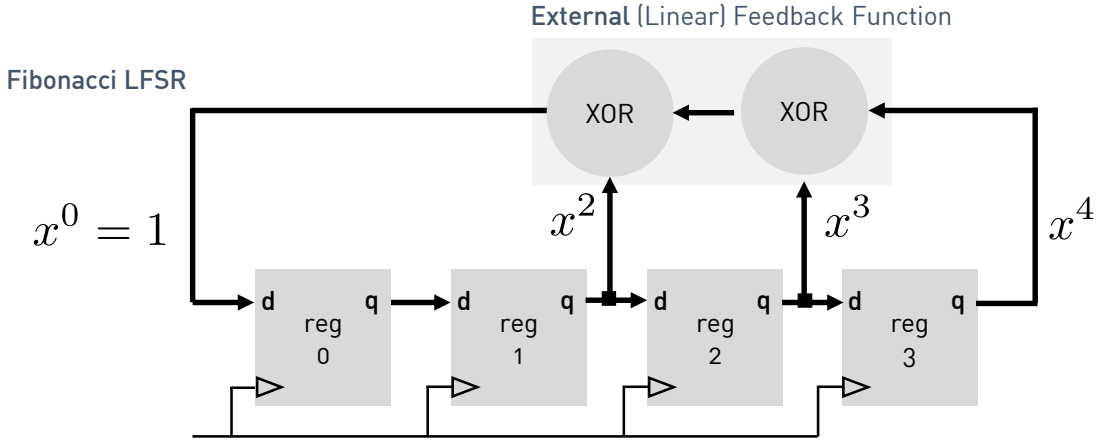
Pseudo 'analog' representation of the discrete LFSR output

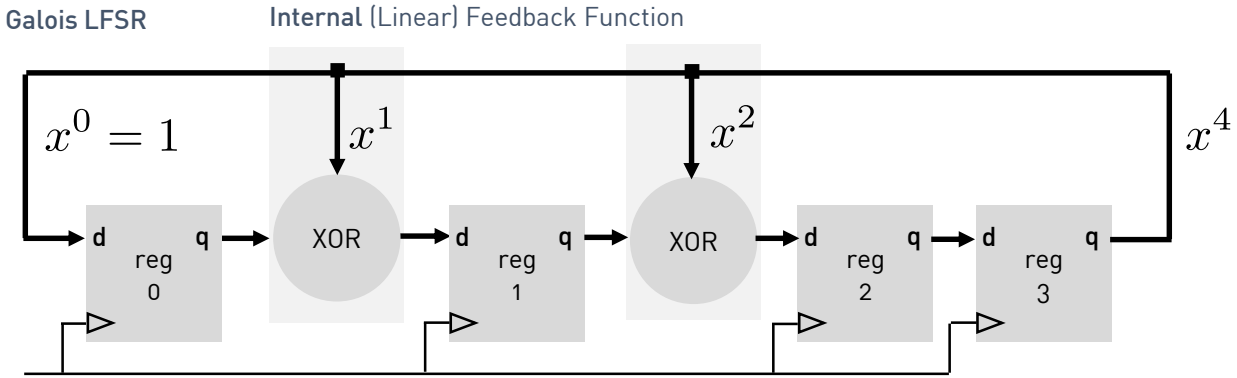
Fibonacci vs. Galois LFSR sequences ...

FPGA based System Design

fbeit
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

XOR based 4-bit Fibonacci LFSR





$$P(x) = x^4 + x^3 + x^2 + 1$$

Fibonacci LFSR Polynomial

Seed				
0	0	1	1	$_3$
0	0	0	1	$_1$
1	0	0	0	$_8$
0	1	0	0	$_4$
1	0	1	0	$_{10}$
1	1	0	1	$_{13}$
0	1	1	0	$_6$

7

$$P(x) = x^4 + x^2 + x^1 + 1$$

Galois LFSR Polynomial

Seed				
0	0	1	1	$_3$
1	1	1	1	$_{15}$
1	0	0	1	$_9$
1	0	1	0	$_{10}$
0	1	0	1	$_5$
1	1	0	0	$_{12}$
0	1	1	0	$_6$

7

Notes

- Non-primitive polynomials
- **The output sequences are different**, however they've got the same sequence-length

XOR based 4-bit Galois LFSR

1/1

```
1.  module lfsr_galois_4bit(
2.     input logic clk, input logic reset_n,
3.     output logic[3:0] q
4.  );
5.
6.  always_ff@(posedge clk)
7.     if(reset_n == 0)
8.        q <= 4'b1001;
11.    else
12.       q <= {q[0],q[3]^q[0],q[2]^q[0],q[1]};
13.
14. endmodule
```

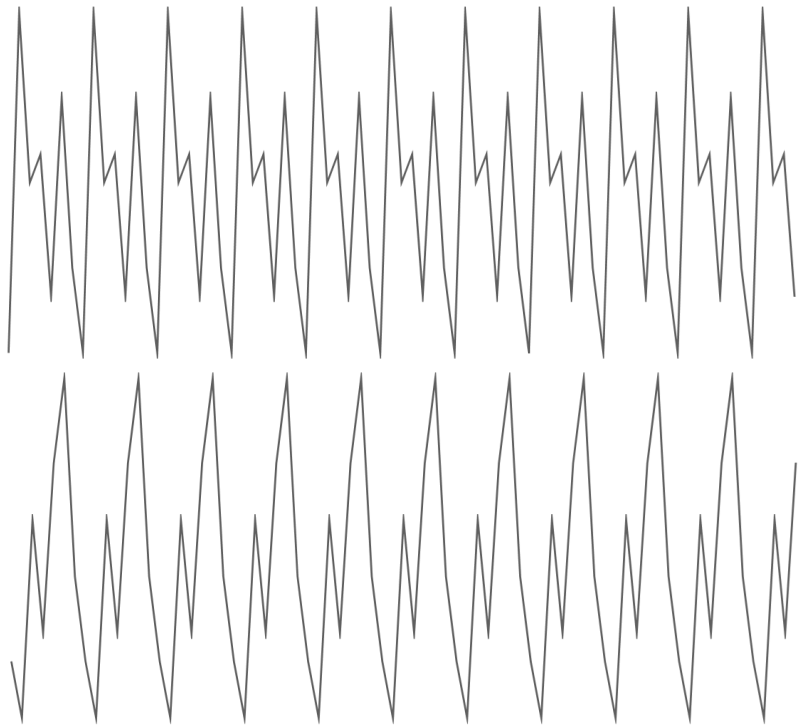
[SystemVerilog] Source Code: lfsr_galois_4bit.sv

$$P(x) = x^4 + x^3 + x^2 + 1$$

Fibonacci LFSR Polynomial

$$P(x) = x^4 + x^2 + x^1 + 1$$

Galois LFSR Configuration



Pseudo 'analog' representation of the discrete LFSR output

How to know if a LFSR polynomial is a primitive polynomial?

FPGA based System Design

Primitive LFSR polynomials with minimum number of XOR gates ...

Bits	Polynomial	Period
2	$P(x) = x^2 + x + 1$	3
3	$P(x) = x^3 + x^2 + 1$	7
4	$P(x) = x^4 + x^3 + 1$	15
5	$P(x) = x^5 + x^3 + 1$	31
6	$P(x) = x^6 + x^5 + 1$	63
7	$P(x) = x^7 + x^6 + 1$	127
8	$P(x) = x^8 + x^6 + x^5 + x^4 + 1$	255
9	$P(x) = x^9 + x^5 + 1$	511
10	$P(x) = x^{10} + x^7 + 1$	1023
11	$P(x) = x^{11} + x^9 + 1$	2047

Notes

- The following table lists maximal-length polynomials for shift-register lengths up to 11. Note that more than one maximal-length polynomial may exist for any given shift-register length.
- A list of alternative maximal-length polynomials for various shift-register lengths can be found in

  www.xilinx.com/support/documentation/application_notes/**xapp052**.pdf

...

153	$P(x) = x^{153} + x^{152} + 1$	$2^{153} - 1$

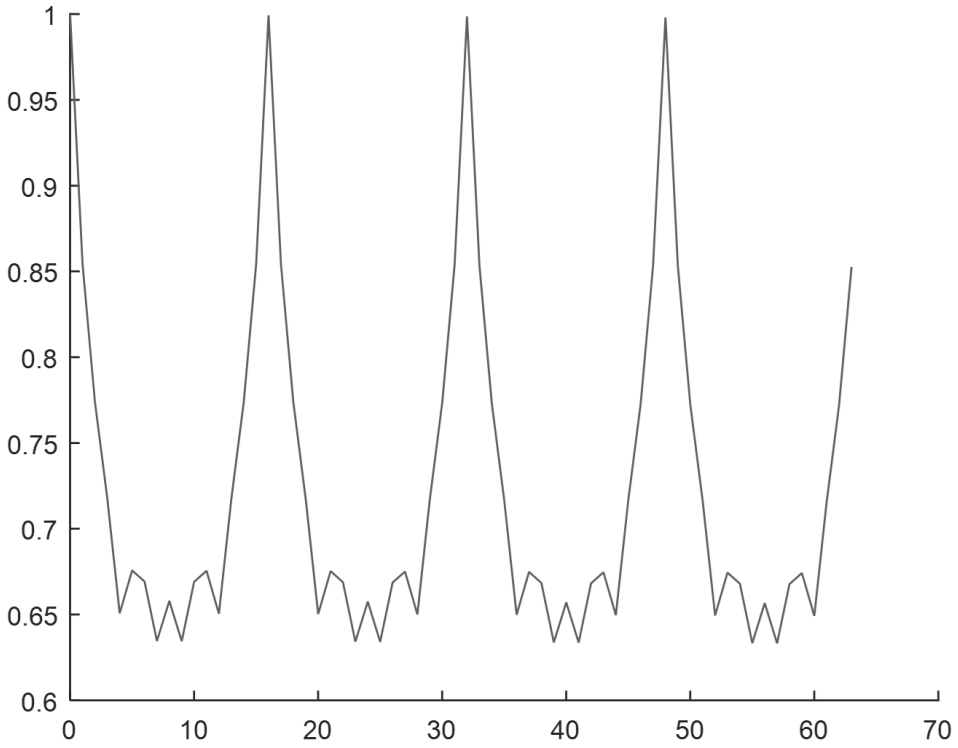
- If this LFSR operates at 50MHz, the sequence first repeats after 7241236391202231765896935410702 7 years ...

How to measure the sequence length of an LFSR?

FPGA based System Design

fbeit
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

Computing the auto-correlation function of an LFSR reveals its sequence length …



Notes

- The discrete auto-correlation function of an N-sample input sequence is defined as:

$$r_{xx}[k] = \sum_{n=0}^{N-1} x[n]x[n-k]$$

- where in hardware, the delays x[n–k] can be implemented using registers, i.e., a delay line containing earlier samples, hence the minus sign in front of k.
- The auto-correlation response is symmetric about the zeroth lag, so in a hardware implementation only the zeroth and positive samples of the autocorrelation need to be calculated

- **LFSR degree?**

How to include the lock-up state into the regular state sequence of an XOR based Fibonacci LFSR ...

FPGA based System Design

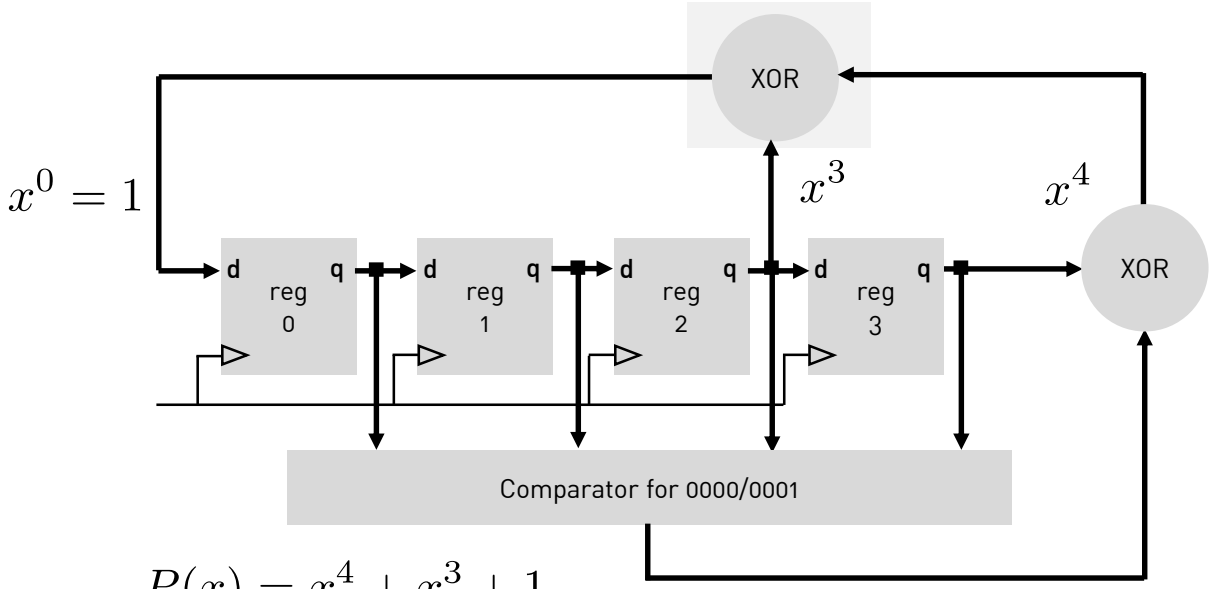
First Solution

FPGA based System Design

How to include the lock-up state into the regular state sequence ...

First solution ....

Linear Feedback Shift Register



$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

Seed	0	0	1	1	3
	0	0	0	1	1
'Lock-up' State	0	0	0	0	0
	1	0	0	0	8
	0	1	0	0	4
	0	0	1	0	2
	1	0	0	1	9
	1	1	0	0	12
	0	1	1	0	6
	1	0	1	1	11
	0	1	0	1	5
	1	0	1	0	10
	1	1	0	1	13
	1	1	1	0	14
	1	1	1	1	15
	0	1	1	1	7
	0	0	1	1	3
	0	0	0	1	1

16

XOR based 4-bit extended Fibonacci LFSR

1/1

```
1.  module ext_lfsr_fibonacci_4bit_comp(
2.     input logic clk, input logic reset_n,
3.     output logic[3:0] q
4.  );
5.  logic lock;
6.  assign lock = ((q == 0) || (q == 1)) ? 1 : 0;
7.  logic feedback; assign feedback = q[0]^q[1]^lock;
8.
9.  always_ff@(posedge clk)
10.    if(reset_n == 0)
11.       q <= 4'b1101;
12.    else
13.       q <={feedback,q[3:1]};
14.
15. endmodule
```

[SystemVerilog] Source Code: ext_lfsr_fibonacci_4bit_comp.sv

$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration

Pseudo 'analog' representation of the discrete LFSR output

Second Solution

FPGA based System Design

fbeit
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

How to include the lock-up state into the regular state sequence …

Second solution …

Linear Feedback Shift Register

Notes

- The all '0's state can't be entered during normal operation but we can get close:

$$0 \quad 0 \quad 0 \quad 1_1$$

- We know that this is a legal state since the only illegal state is all 0's. If the first 3 bits are '0', then bit 0 must be a '1'.
- Now, since the XOR inputs are a function of taps, including the bit 0 tap, we know what the output of the XOR function will be! It must be a '1'. So normally the next state will be:

$$1 \quad 0 \quad 0 \quad 0_8$$

$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

How to include the lock-up state into the regular state sequence …

Second solution …

Linear Feedback Shift Register

Notes

- So, in order to include the actual illegal lock-up state, start with detecting the "**almost**" lock-up state:

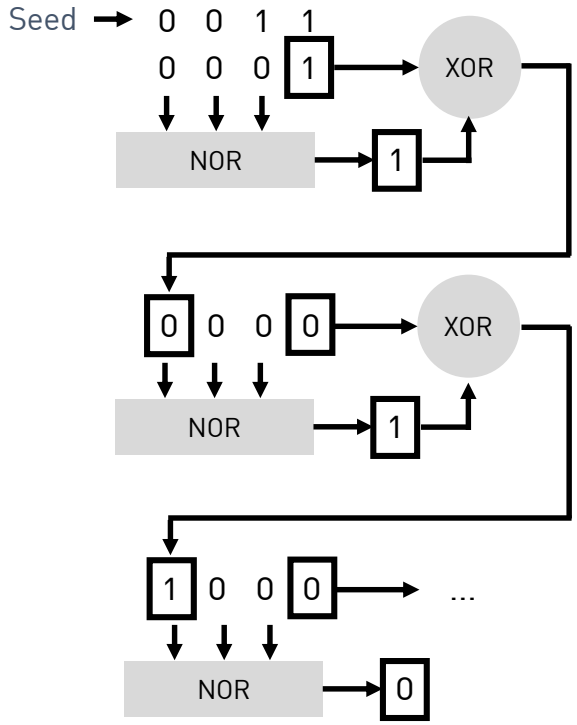
  0 0 0 x

- Computing the Boolean NOR function for the first three will provide a '1' when they are all '0. We add this term to the regular XOR feedback function …

$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

Extended XOR based 4-bit Fibonacci LFSR



$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

Notes

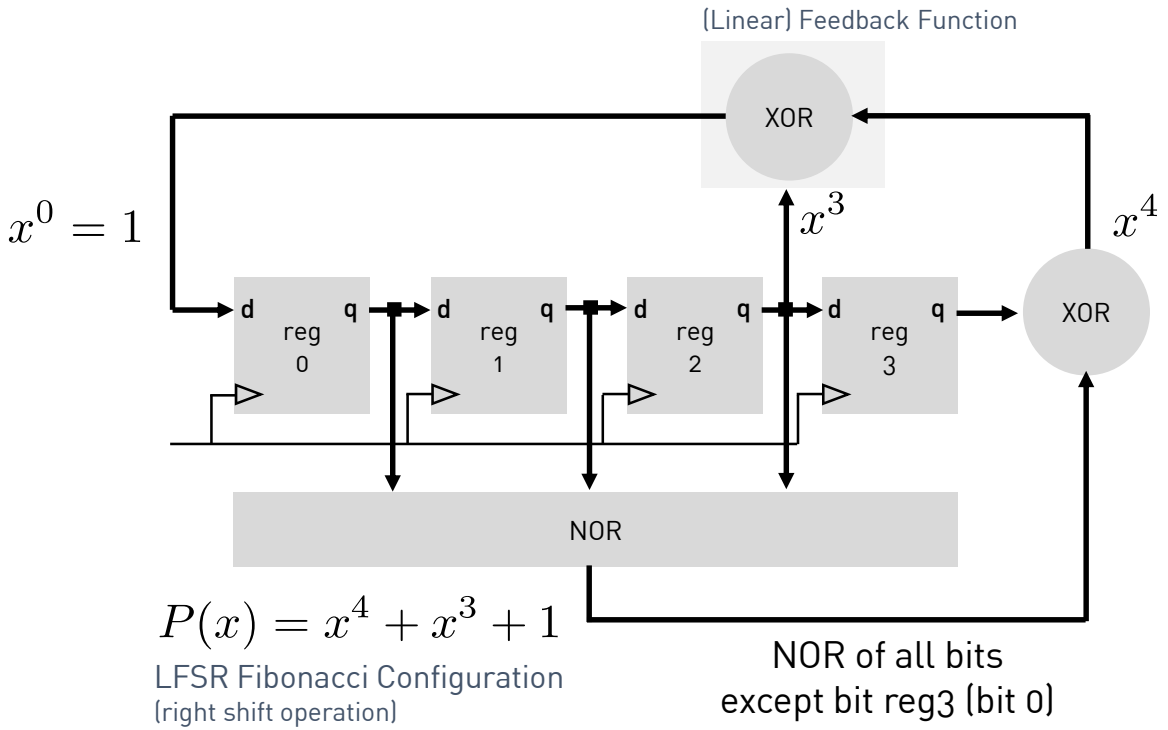
- **The NOR gate generates a logic high output only for an all-zero input.**
- This only applies for the input patterns q = 4'b0000 and 4'b0001.
- In the present case, the output of the NOR gate acts as a control signal:

ctrl	d	q
0	0	0
0	1	1

ctrl	d	q
1	0	1
1	1	0

- **For ctrl being low**, the input signal d is just passed to the output q, whereas **when ctrl is high**, the input is inverted and passed to the output.
- **The XOR gate therefore acts as an inverter that can be activated or deactivated**.

Extended XOR based 4-bit Fibonacci LFSR



Seed	0 0 1 1	3
	0 0 0 1	1
'Lock-up' State	0 0 0 0	0
	1 0 0 0	8
	0 1 0 0	4
	0 0 1 0	2
	1 0 0 1	9
	1 1 0 0	12
	0 1 1 0	6
	1 0 1 1	11
	0 1 0 1	5
	1 0 1 0	10
	1 1 0 1	13
	1 1 1 0	14
	1 1 1 1	15
	0 1 1 1	7
	0 0 1 1	3
	0 0 0 1	1

16

XOR based 4-bit extended Fibonacci LFSR

1/1

```
1.  module lfsr_galois_4bit_nor(
2.     input logic clk, input logic reset_n,
3.     output logic[3:0] q
4.  );
5.
6.  logic tmp;      assign tmp      = ~(q[1]|q[2]|q[3]);
7.  logic feedback; assign feedback = q[0]^q[1]^tmp;
8.
9.  always_ff@(posedge clk)
10.    if(reset_n == 0)
11.       q <= 4'b1101;
12.    else
13.       q <={feedback,q[3:1]};
14.
15. endmodule
```

[SystemVerilog] Source Code: ext_lfsr_fibonacci_4bit_nor.sv

$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration

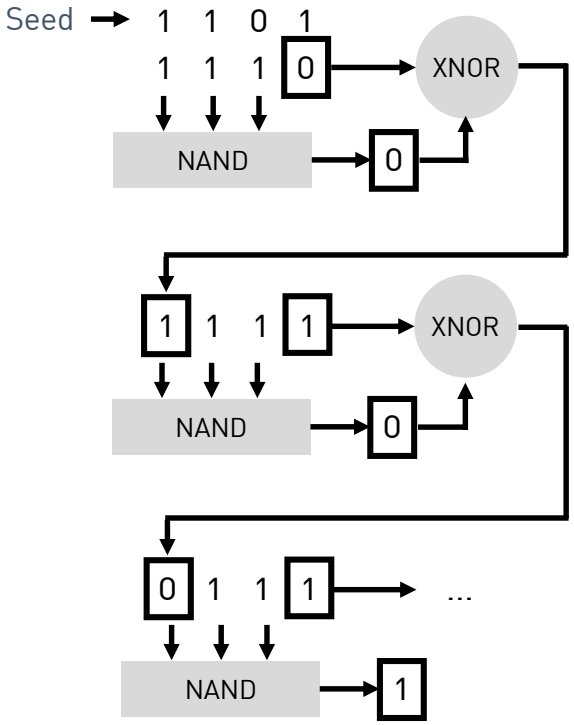
Pseudo 'analog' representation of the discrete LFSR output

How to include the lock-up state into the regular state sequence of an XNOR based Fibonacci LFSR ...

FPGA based System Design

fbeit
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

Extended XNOR based 4-bit Fibonacci LFSR



$$P(x) = x^4 + x^3 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

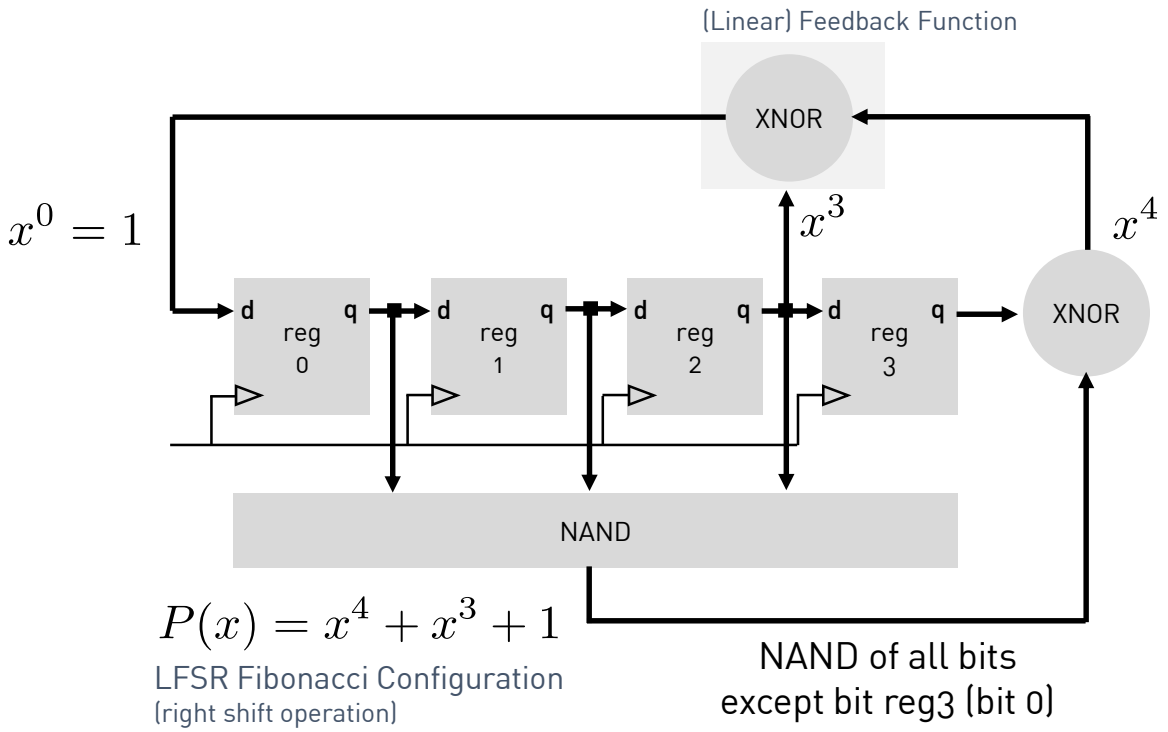
Notes

- Now, the all **one's state** represents the lock-up state.
- The NAND gate generates a logic high output only for an all-ones input.
- This only applies for the input patterns q = 4'b1110 and 4'b1111.
- In the present case, the output of the NAND gate acts as a control signal:

ctrl	d		q	ctrl	d		q
0	0	→	1	1	0	→	0
0	1		0	1	1		1

- For ctrl being low, the input signal d is inverted and passed to the output q, whereas when ctrl is high, the input is just passed to the output.
- The XNOR gate therefore acts as an inverter that can be activated or deactivated.

Extended XNOR based 4-bit Fibonacci LFSR



Seed	0	0	1	1	3
	0	0	0	1	1
	0	0	0	0	0
	1	0	0	0	8
	0	1	0	0	4
	0	0	1	0	2
	1	0	0	1	9
	1	1	0	0	12
	0	1	1	0	6
	1	0	1	1	11
	0	1	0	1	5
	1	0	1	0	10
	1	1	0	1	13
	1	1	1	0	14
'Lock-up' State	1	1	1	1	15
	0	1	1	1	7
	0	0	1	1	3
	0	0	0	1	1

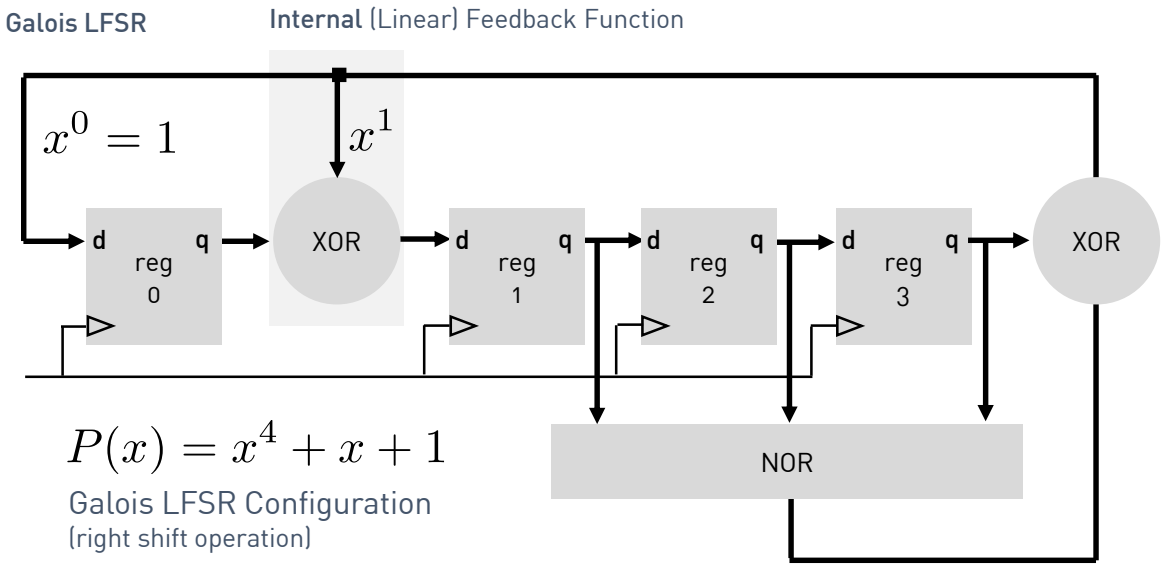
16

How to include the lock-up state into the regular state sequence of an XOR/XNOR based Galois LFSRs ...

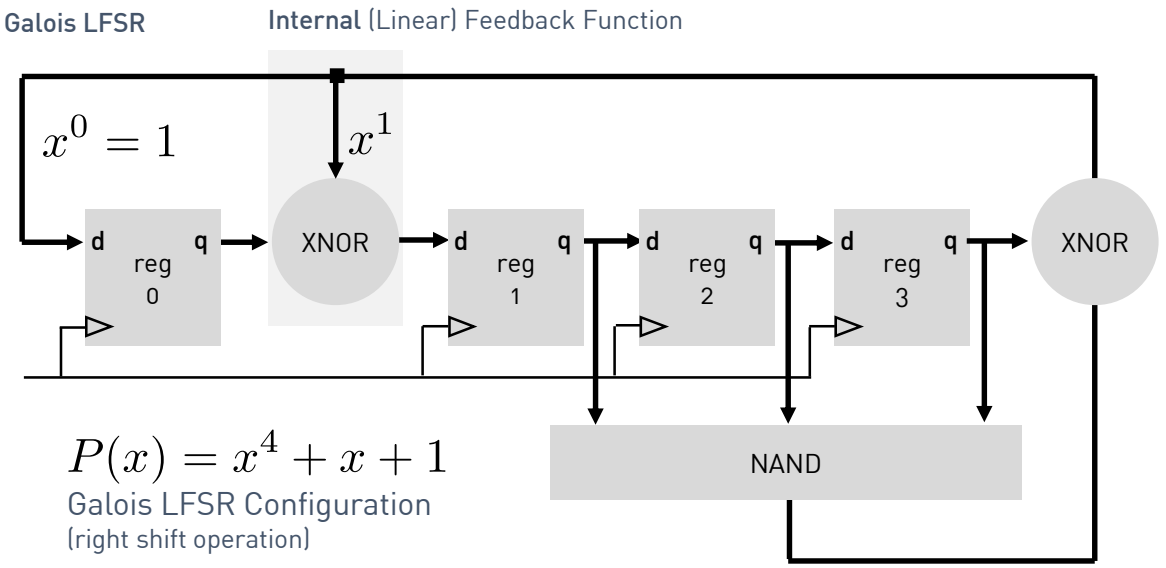
FPGA based System Design

fbeit
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

XOR based 4-bit Galois LFSR



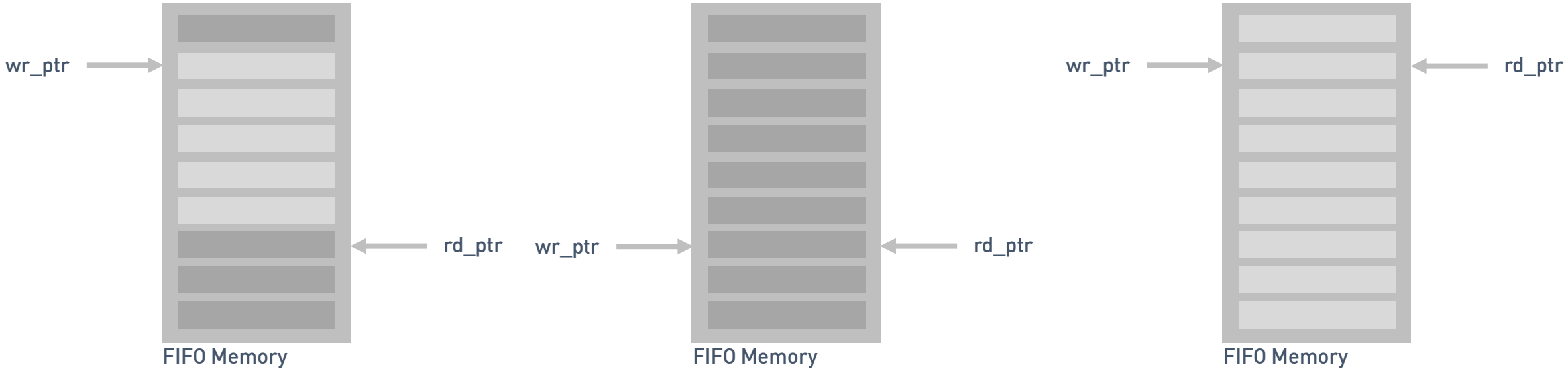
XNOR based 4-bit Galois LFSR



LFSR's in Dual Clock (DC) First in First Out (FIFO) Memories ...

FPGA based System Design

LFSR's in Single Clock (SC) First in First Out (FIFO) memories

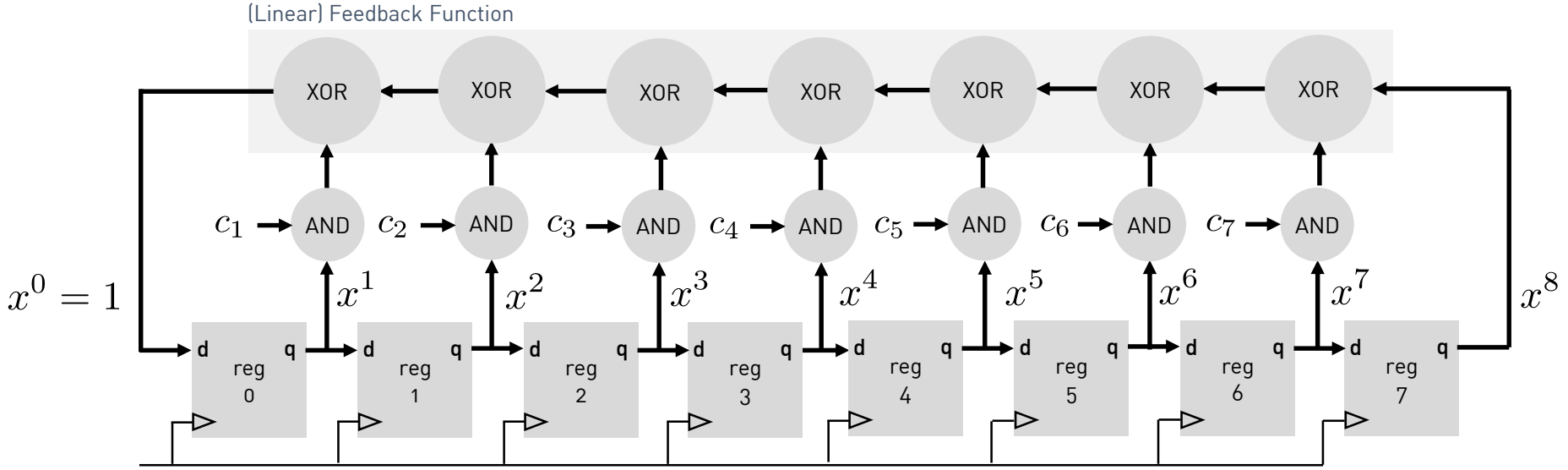


- Address pointers are used internally to keep the next write and read position within the dual-port memory
- **Advantage of using an LFSR counter?**
- If both pointers have the same values after a write operation, the FIFO is full

Configurable LFSR structure ...

FPGA based System Design

Configurable LFSR structure ...



$$P(x) = x^8 + c_7x^7 + c_6x^6 + c_5x^5 + c_4x^4 + c_3x^3 + c_2x^2 + c_1x^1 + 1$$

LFSR Fibonacci Configuration
(right shift operation)

LFSRs as Pseudo Random Number Generators (PRNG)

FPGA based System Design

fbeit
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

LFSRs as Pseudo Random Number Generators (PRNG)

Pseudo 'analog' representation of the discrete LFSR output

Notes

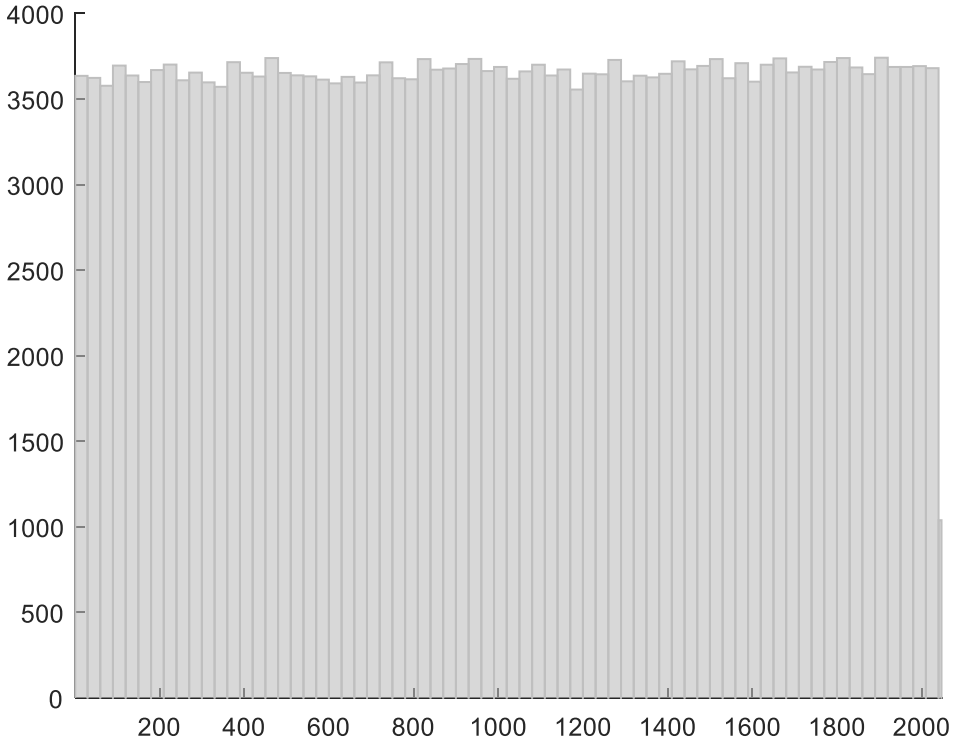
- For what type of applications, pseudo random numbers are not sufficient any more and why?

LFSRs as Pseudo Random Number Generators (PRNG) with predefined distribution

FPGA based System Design

fbeit
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

LFSRs as Pseudo Random Number Generators (PRNG) with predefined distribution



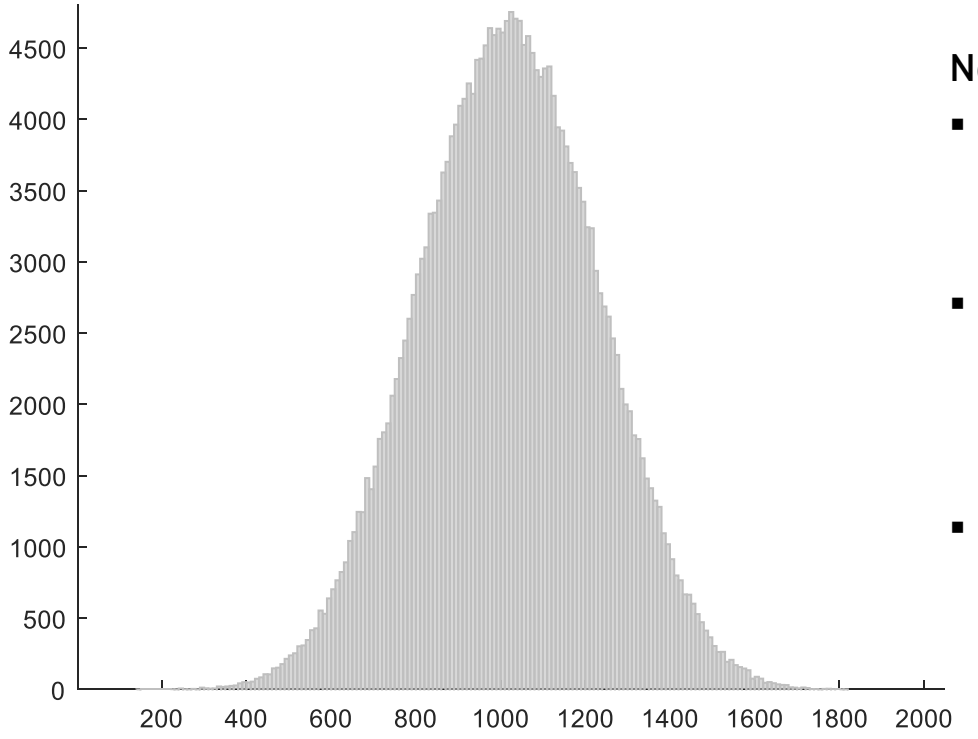
Notes

- Linear Feedback Shift Registers produce a sequence of numbers that appears to be uniformly distributed over the range 1 to $2^n - 1$, where n is the LFSR size. For this type of generator, each output value has an **equal probability of appearing (uniform distribution)**.

Question

- So I know how to generate a uniform random variable. How do I generate others?

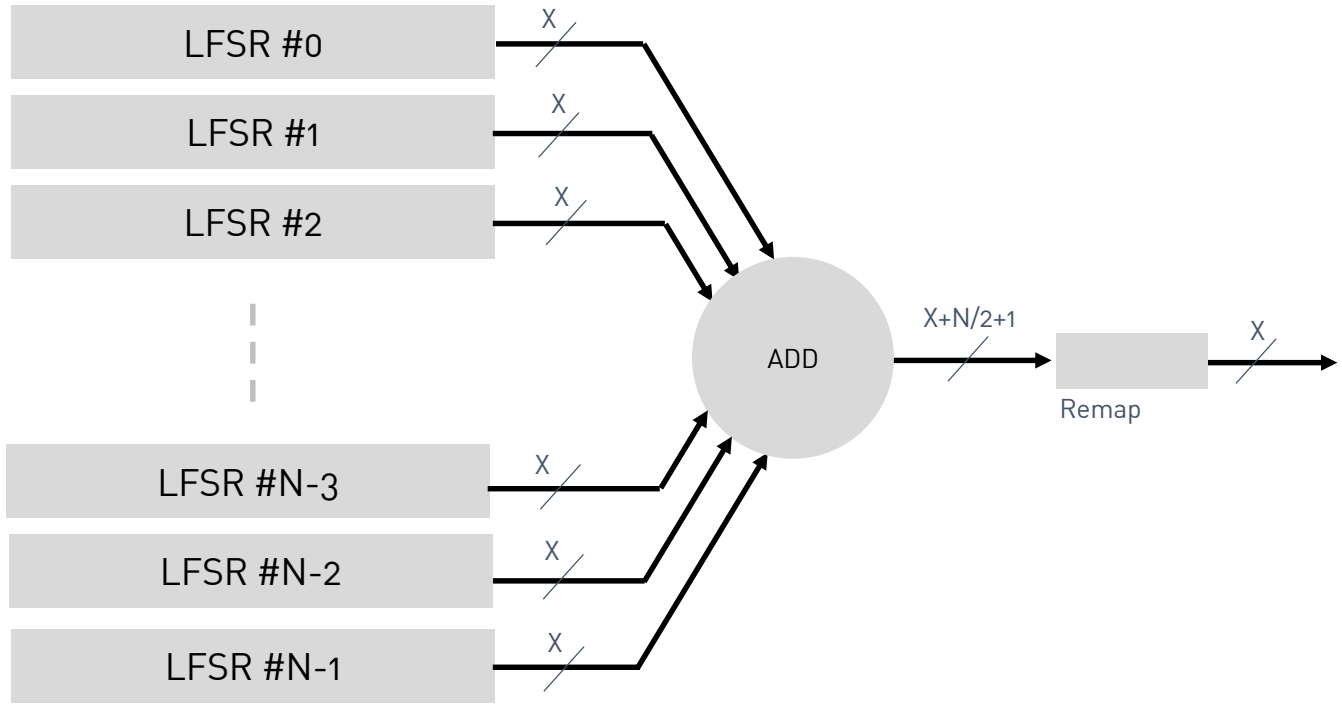
LFSRs as Pseudo Random Number Generators (PRNG) with predefined distribution

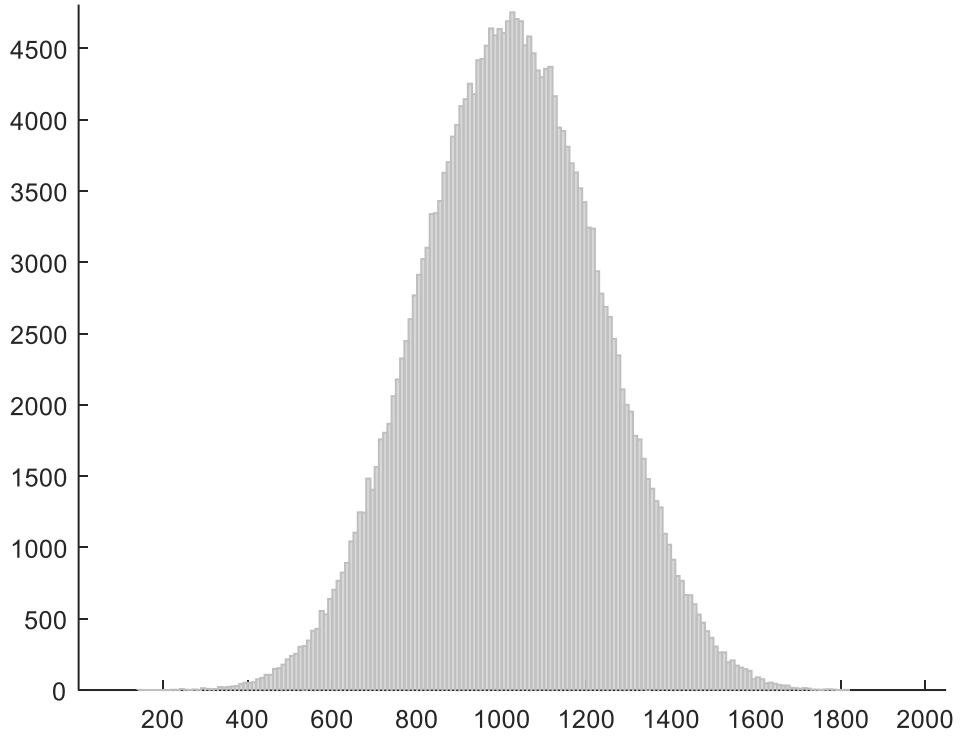


Notes

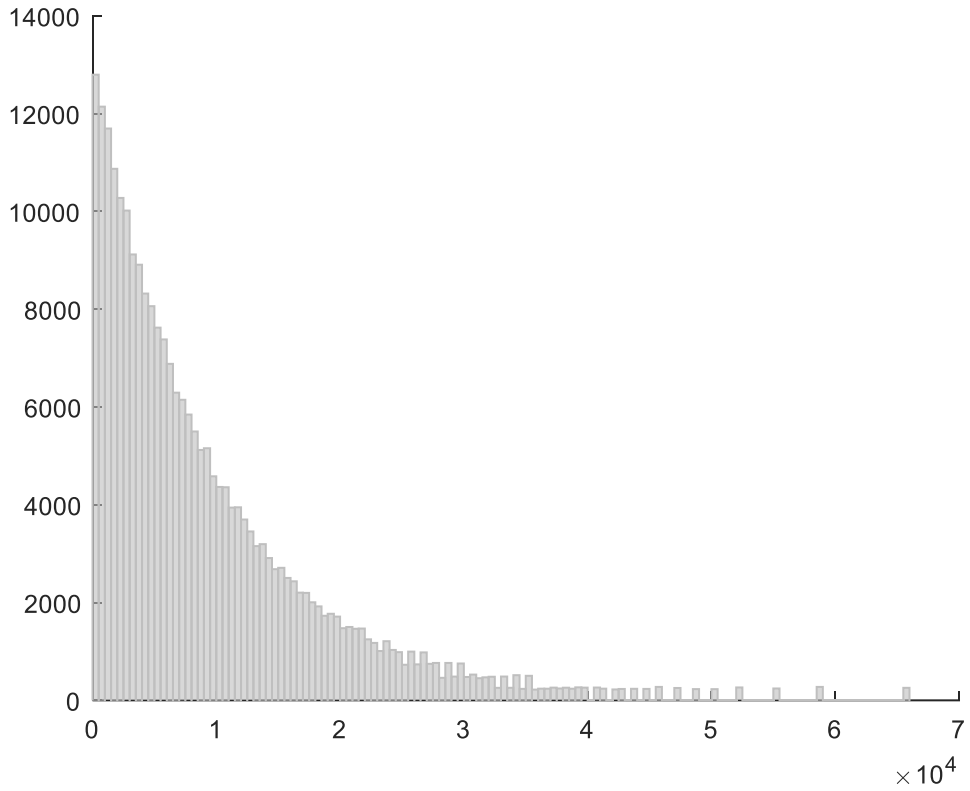
- Some applications require the generation of random numbers that aren't uniformly distributed. In many DSP applications, it is often necessary to produce values that are **normally distributed**. One of the most popular schemes to generate normal distributed pseudo random numbers is based on the Central Limit Theorem (CLT).
- In probability theory, the **Central Limit Theorem CLT** states that, given certain conditions, the arithmetic mean of a sufficiently large number of iterates of independent and identically distributed random variables will be approximately normally distributed. Or in other words: The sum of N random numbers will approach a normal distribution as N approaches infinity.
- Once we can generate pseudo random sequences with normal distribution, we can also adjust the **signal's mean** and **variance** to use it for any purpose.

LFSRs as Pseudo Random Number Generators (PRNG) with predefined distribution





LFSRs as Pseudo Random Number Generators (PRNG) with predefined distribution



Notes

- Consider an **exponential distribution**: the density of an exponential random variable X is given by

$$f_X(x) = \begin{cases} e^{-x} & x \geq 0 \\ 0 & x < 0\,. \end{cases}$$

- Thus, its **distribution function** is given by

$$F_X(x) = 1 - e^{-x}$$

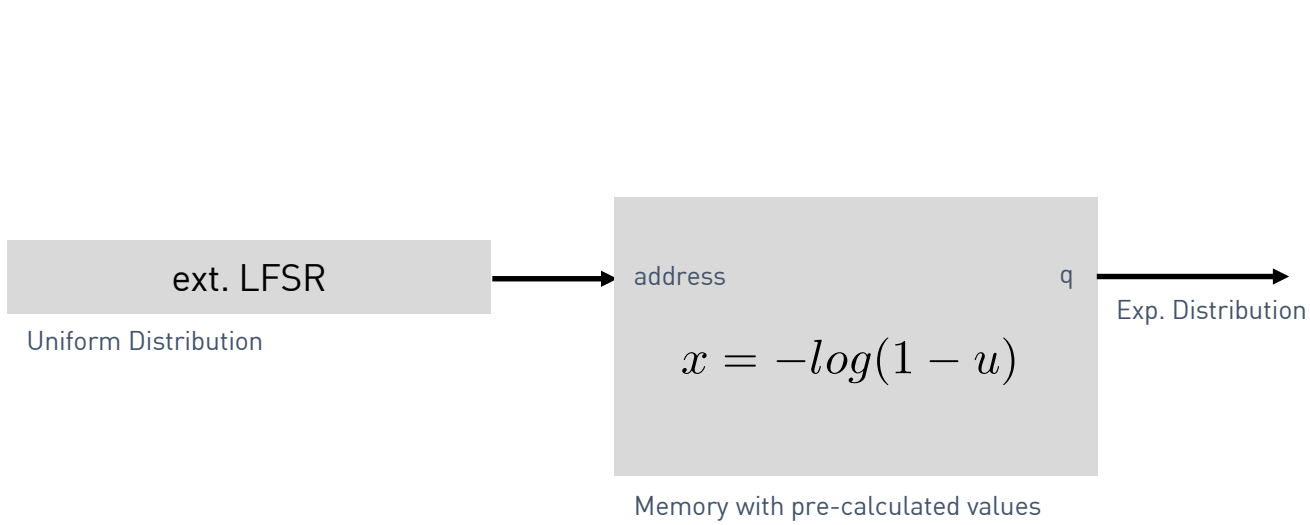
- Let us determine the **inverse to** $F_X(x)$. We need to solve for x in

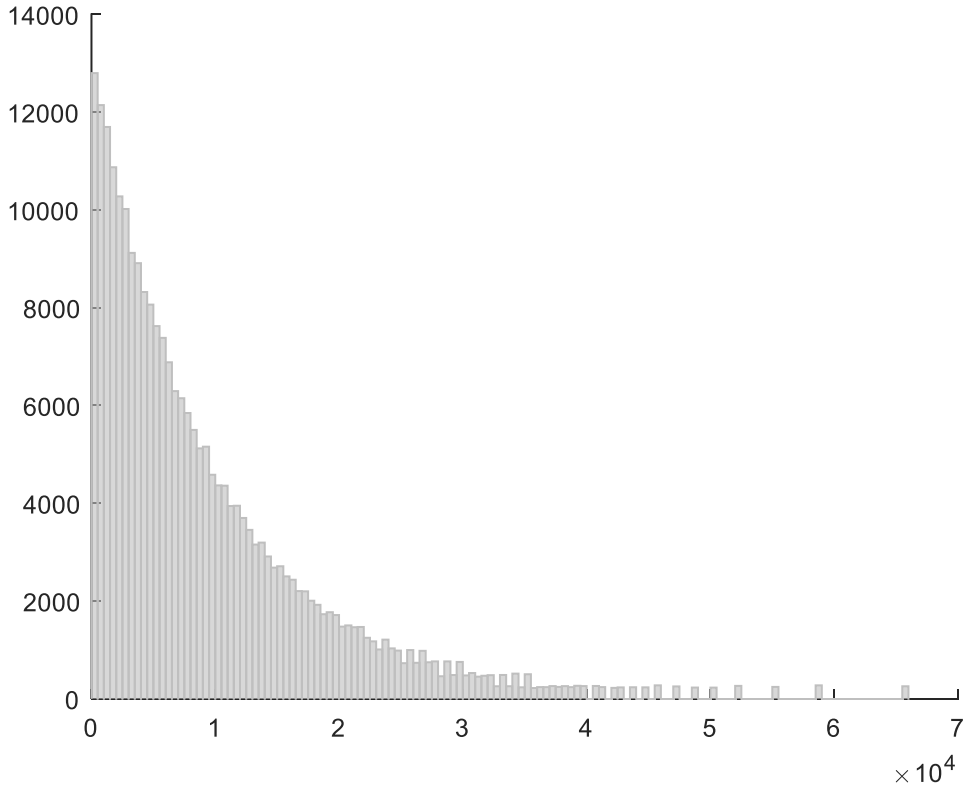
$$1 - e^{-x} = u$$

- Thus,

$$x = -log(1 - u)$$

LFSRs as Pseudo Random Number Generators (PRNG) with predefined distribution





Built-In Self-Test (BIST) - Generating bit streams with defined bit probabilities

FPGA based System Design

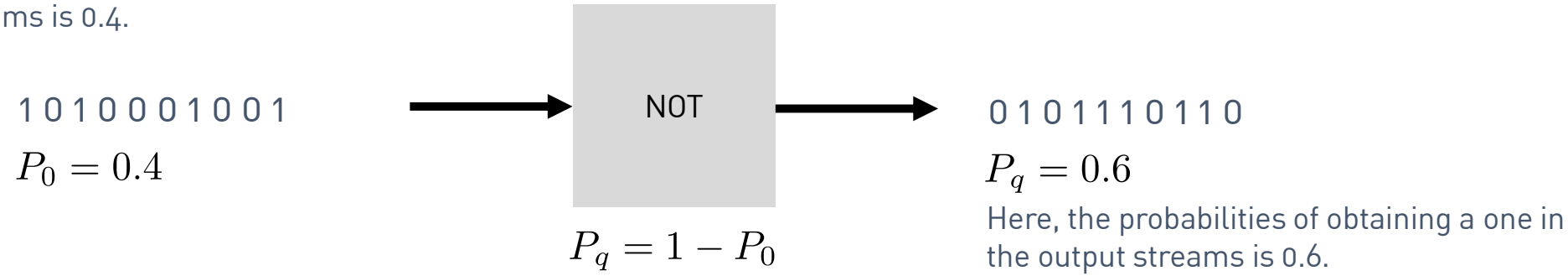
fbeit
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

Transforming Bit Probabilities with Combinational Logic

- The intention is to generate bit sequences with random bit density.
- We are using LFSRs in conjunction with combinational logic circuits to transform bit streams with given source probabilities into bit streams with different target probabilities.

Transforming Bit Probabilities with Combinational Logic

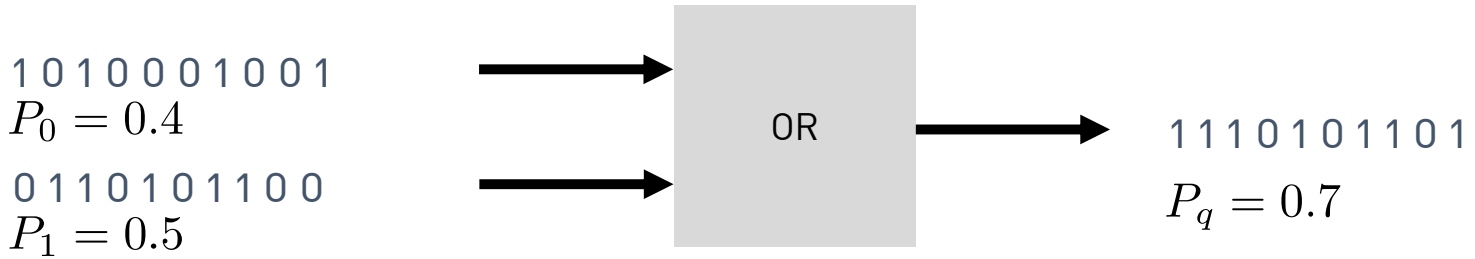
Here, the probabilities of obtaining a one in the input streams is 0.4.



Notes

- With probability p, we mean "with a probability of p being at logic one".
- Given an input with probability 0.4, an inverter will have an output z with probability 0.6 since: $P_q = 1 - P_0$

Transforming Bit Probabilities with Combinational Logic

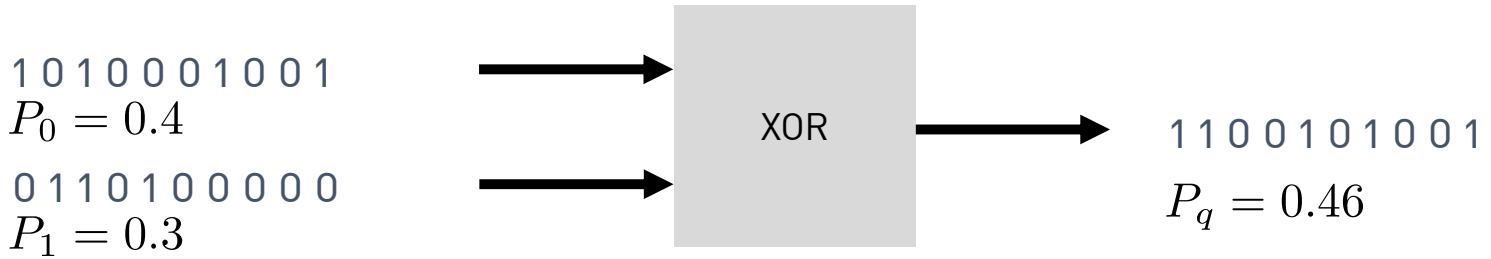


$$P_q = 1 - (1 - P_0) \times (1 - P_1)$$

Notes

- With probability p, we mean "with a probability of p being at logic one"
- **Homework**: Derive the upper equation for a logical OR gate

Transforming Bit Probabilities with Combinational Logic

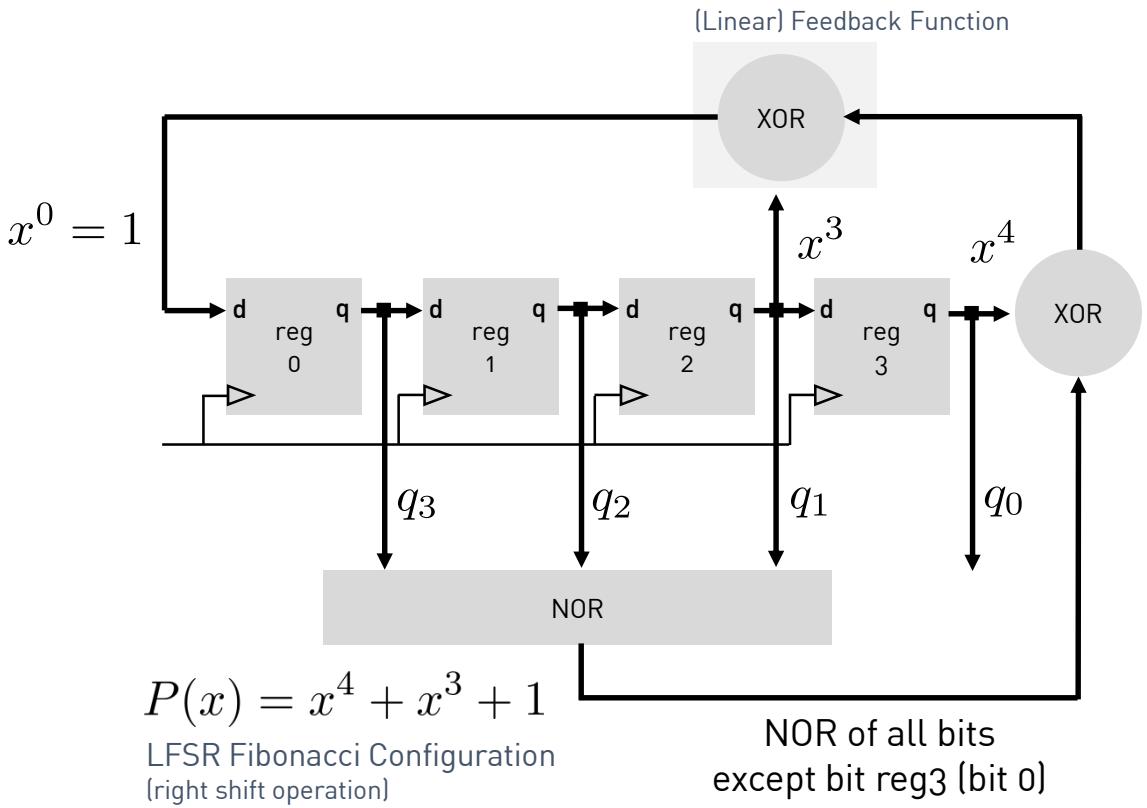


$$P_q = 1 - (1 - P_0) \times P_1 + (1 - P_1) \times P_0$$

Notes

- With probability p, we mean "with a probability of p being at logic one"

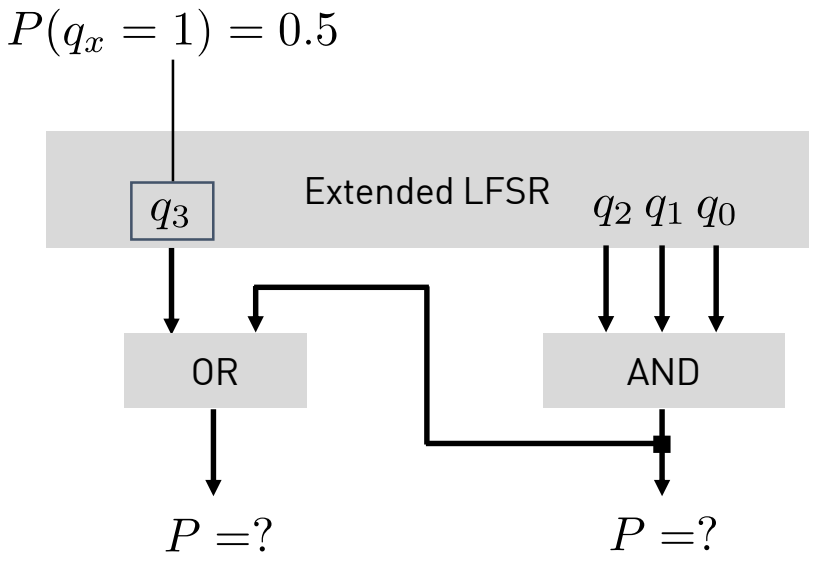
Generating bit streams with defined bit probabilities



	q_3	q_2	q_1	q_0	
Seed	0	0	1	1	3
	0	0	0	1	1
'Lock-up' State	0	0	0	0	0
	1	0	0	0	8
	0	1	0	0	4
	0	0	1	0	2
	1	0	0	1	9
	1	1	0	0	12
	0	1	1	0	6
	1	0	1	1	11
	0	1	0	1	5
	1	0	1	0	10
	1	1	0	1	13
	1	1	1	0	14
	1	1	1	1	15
	0	1	1	1	7

16

At every output stage:
$8 \times 1 \quad \& \quad 8 \times 0$



How to test these structures?

FPGA based System Design